

API 文件前置作業

目錄

- 1.1 API 認證
- 1.2 API 共通規範
- 1.3 狀態碼
- 1.4 轉真人 Webhook 事件
- 1.5 客戶端工具 (Client Tools)

API 認證

MaiAgent 使用 API 金鑰進行身分驗證。

身分認證方式

所有 API 請求必須在 HTTP Header 中包含 API Key，格式如下：

```
const headers = {  
  "Authorization": "Api-Key YOUR_API_KEY_HERE",  
  "Content-Type": "application/json"  
}
```

標頭的前綴是 **Api-Key**，不是 **Bearer**。金鑰綁定哪位成員、權限怎麼算、什麼情況會失效等規則，見 [API 共通規範：認證](#)。

如何獲取 API 金鑰？

API 金鑰有兩種取得方式，兩種金鑰的呼叫方式相同：

成員自建：組織內的有效成員都可以在後台右上角頭像 → **帳號** → **API 金鑰** 為自己建立金鑰，不需要額外權限。

管理者代發：具備「API Key 管理權限」的管理者，可以在 **組織設定** → **API Key 管理** 替指定成員簽發金鑰，並設定到期時間與每分鐘請求數（RPM）。

操作步驟見使用者手冊 [API Key 管理](#)：[成員自行建立金鑰](#)、[管理者建立 API Key](#)。

安全提醒：請妥善保管您的 API 金鑰，不要在公開場所分享 金鑰格式說明：API 金鑰由前綴與密鑰兩段英數字組成，中間以 . 相連，呼叫時請完整帶入

完整金鑰只會在建立成功時顯示一次。之後的列表僅顯示部分字元，無法再次查看或還原完整金鑰；若未保存，請建立新金鑰、更新使用該金鑰的服務，再撤銷舊金鑰。

找不到舊的系統簽發金鑰

個人帳號的 **API 金鑰** 頁面只列出您自行建立的金鑰。首次加入組織（包含透過 AD 首次登入）時，系統可能會自動簽發一把無到期日的金鑰，但不會提供其完整值，因此無法取得該金鑰來呼叫 API。若要呼叫 API，請在個人頁面自行建立新金鑰，並在建立成功時立即保存完整值。

系統自動簽發或由管理者代為簽發的金鑰，只有具有組織金鑰管理權限的成員可從 **組織金鑰管理** 查看其擁有者、前綴、簽發來源、狀態與到期日，但該頁不會顯示完整值。若您沒有此權限，可請組織擁有者或具有組織金鑰管理權限的成員協助確認或撤銷舊金鑰。

API 金鑰的權限範圍

以 API 金鑰呼叫 API 時，權限與金鑰擁有者在該組織的角色權限相同；缺少所需權限時，API 會回傳 403 Forbidden。

成員 (members) 與角色 (groups) 的寫入操作都需要子權限 **組織概覽權限** (位於 **組織權限** 之下)，組織擁有者則不受此限。適用的操作包含：

新增成員： `POST /api/v1/organizations/{organizationPk}/members/`

移除成員： `DELETE /api/v1/organizations/{organizationPk}/members/{id}`

調整成員所屬角色： `POST /api/v1/organizations/{organizationPk}/members/{id}/update-groups/`

建立、更新、刪除角色： `POST`

`PUT`

`PATCH`

`DELETE /api/v1/organizations/{organizationPk}/groups/...`

批量新增或移除角色成員、分配或移除角色的 AI 助理、收件匣、知識庫與資料

庫： `/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-*`

`...`

其他成員與角色的寫入操作 (例如批次匯入成員) 同樣適用

權限以子項判斷：角色設定中父層 **組織權限** 未勾，不代表其下的 **組織概覽權限** 未勾。排查 403 時，請到後台 **組織設定 → 角色** 展開 **組織權限**，確認金鑰擁有者所屬角色是否勾選 **組織概覽權限**。

錯誤處理

當認證錯誤時，API 會回傳 401 Unauthorized 的 HTTP 錯誤代碼：

API Key 缺失、格式錯誤或無效

沒帶金鑰、標頭格式不對，或金鑰無效 (不存在、已到期、已停用、已撤銷，或綁定的成員已停用、已移出組織)，都回同樣的內容，不會說明是哪一種原因：

```
{
  "detail": "Authentication credentials were not provided."
}
```

修復建議：檢查請求標頭是否為 `Authorization: Api-Key YOUR_API_KEY`，並確認 API Key 完整、沒有多餘的空格或字元。格式正確仍失敗時，請到 [API Key 管理](#) 確認金鑰不是已停用、已過期或已撤銷，以及綁定的成員仍是啟用狀態。

把 API Key 放在 `Bearer` 後面

平台會把它當成登入權杖驗證，回應內容不同：

```
{
  "detail": "Given token not valid for any token type",
  "code": "token_not_valid",
  "messages": [ ... ]
}
```

修復建議：把前綴改成 `Api-Key`。

401 回應會帶 `WWW-Authenticate` 標頭，多數端點的值是 `Bearer realm="api"`，請以狀態碼判斷，不要依這個標頭決定要帶哪一種憑證，說明見 [API 共通規範：認證失敗的回應](#)。金鑰有效但權限不足時回 **403**，見上方 [API 金鑰的權限範圍](#)。

API 共通規範

本頁整理所有平台 API 共用的規則：網址、認證、限流、錯誤回應、分頁、欄位格式與上傳上限。各端點的欄位與範例見左側「API 端點說明文件」，本頁的規則適用於全部端點，端點頁不再重複。建議在開始寫程式前讀過一次。

建立與管理 API Key 的畫面操作見使用者手冊 [API Key 管理](#)；查呼叫紀錄見 [API 呼叫紀錄](#)。從零開始串接的步驟見技術人員手冊 [快速上手](#)。

網址與版本前綴

SaaS 的 API 網址是 <https://api.maiagent.ai/api/v1/>；地端與私有雲部署請換成貴公司的平台網域。

`/api/` 與 `/api/v1/` 掛的是同一套端點，路徑、欄位與權限都相同。

AI Gateway（OpenAI 相容端點）與 SCIM 只在 `/api/v1/` 底下。

新的整合請一律使用 `/api/v1/`。

端點路徑以

結尾。少了結尾的

會收到 `301` 轉址到加上

的網址；部分 HTTP 用戶端跟隨轉址時會把 `POST` 改成 `GET`，請直接呼叫帶的網址。

認證

伺服器對伺服器的呼叫使用 API Key，放在 `Authorization` 標頭：

```
Authorization: Api-Key <您的 API Key>
```

金鑰由前綴與密鑰兩段英數字組成，中間以 `.` 相連，請完整帶入。

規則	說明
金鑰綁定一位成員	呼叫時的組織就是這位成員所屬的組織，請求不需要另外帶組織 ID
權限等於該成員的角色	沒有依端點細分的 scope；成員能做的事，金鑰都能做。缺少權限回 <code>403</code> ，見 API 金鑰的權限範圍
成員停用即失效	成員被停用或移出組織後，這把金鑰立即無法使用

規則	說明
到期、停用、撤銷	和不存在的金鑰一樣回 <code>401</code> ，回應不會說明原因
完整金鑰只顯示一次	建立後只會顯示部分字元，遺失請重新建立

金鑰的建立與管理入口見 [API 認證方式](#)。

認證失敗的回應

沒帶金鑰、標頭格式不對，或金鑰無效（不存在、已到期、已停用、已撤銷，或綁定的成員已停用、已移出組織）都回 `401`，內容相同：

```
{"detail": "Authentication credentials were not provided."}
```

金鑰無效時，訊息同樣是「沒有提供認證資訊」：平台把認不得的金鑰當成沒有帶憑證，不會說明是哪一種原因。

回應帶 `WWW-Authenticate` 標頭。多數端點的值是 `Bearer realm="api"`（這些端點也接受 OAuth 存取權杖）；只接受 API Key 的端點（例如下方的 `api-key-context`）是 `Api-Key`。請以 `401` 狀態碼判斷，不要依這個標頭的值決定要帶哪一種憑證。

把 API Key 放在 `Bearer` 後面，平台會把它當成登入權杖驗證，回 `401` 與不同的內容：

```
{"detail": "Given token not valid for any token type", "code": "token_not_valid", "messages": [ ... ]}
```

收到這個回應時，請把標頭改成 `Api-Key <金鑰>`。

整合用金鑰的建議：建一位專用的整合成員，只給它需要的角色權限，再用它的身分簽發金鑰並設定到期時間。綁在個人帳號上的金鑰，會在這位員工離職、帳號被停用時一起失效，整合也跟著中斷。API Key 請只放在伺服器端：平台 API 只對允許的網域開放跨來源（CORS）呼叫，也不應該讓金鑰出現在瀏覽器裡。

確認金鑰所屬的組織

要在自家閘道先確認金鑰屬於哪個組織，可以呼叫 `GET /api/v1/users/api-key-context/`，回應 `{"organizationId": "..."}` 。這支端點只接受 API Key，不計入金鑰的 RPM。

其他憑證

讓 MCP 客戶端以成員身分連線，見技術人員手冊 MaiAgent 作為 OAuth 授權伺服器；在自家系統嵌入 MaiGPT 的成員權杖，見 串接 MaiGPT：成員身分嵌入。

速率限制

平台 API **沒有全域的預設上限**。會限流的只有下表這些：

範圍	上限	計算方式	誰可以調整
每把 API Key 的請求數（RPM）	管理者設定，未設定時不限	每把金鑰、每 60 秒	組織的 API Key 管理
AI Gateway 的 Token 數（TPM）	管理者設定，未設定時不限	每把金鑰、每分鐘，只算 Gateway 推論	同上，見 <u>AI Gateway 的限制</u>
SCIM 端點	100 次／分鐘	每個組織	固定
<code>web-chats/{id}/setup-contact-credentials/</code>	30 次／分鐘	每個來源 IP	固定
免登入的計價查詢（ <code>credit/items/</code> 、 <code>credit/pricing-rules/</code> ）	60 次／分鐘	每個來源 IP	固定
<code>auth/login/</code>	預設 10 次／分鐘	每個來源 IP	平台設定
不帶憑證的 Web Chat 請求（接續對話、上傳附件等）	10~30 次／分鐘，依端點而定	每個來源 IP	固定

金鑰的 RPM 作用在這把金鑰能呼叫的平台 API，不只 AI Gateway。表中有固定上限的端點使用自己的上限，不計入金鑰的 RPM。

OAuth 存取權杖與後台登入的請求不受金鑰 RPM 限制。

超過上限回 `429`，並帶 `Retry-After` 標頭（秒數）。請依這個秒數退避後再重試，不要立刻重送。

```
{"detail": "Request was throttled. Expected available in 42 seconds."}
```

錯誤回應

平台 API 沒有統一的錯誤碼表。處理錯誤時請依 **HTTP 狀態碼**判斷，回應帶 `code` 時再依 `code` 細分；**不要比對 `detail` 文字**：文字會隨 `Accept-Language` 翻譯，也可能調整措辭。

回應內容的形狀

形狀	範例	常見於
<code>detail</code> 物件	<code>{"detail": "You do not have permission to perform this action."}</code>	認證、權限、找不到、限流、方案與 Credits；少數會多帶 <code>code</code> 或其他欄位
欄位物件	<code>{"name": ["This field is required."]}</code>	欄位驗證失敗；key 是 camelCase 欄位名，不屬於單一欄位的錯誤放在 <code>nonFieldErrors</code>
字串陣列	<code>["..."]</code>	少數沒有對應欄位的驗證錯誤

例外情況：

AI Gateway 的 OpenAI 相容端點使用 OpenAI 格式的 `error` 物件，見 [AI Gateway 錯誤格式](#)。

SCIM 端點的請求內容錯誤、找不到與衝突，依 SCIM 2.0 規格回應（`schemas`、`status`、`detail`）。

網址不存在（路徑打錯、少了資源 ID）時回 `404`，內容是 HTML，不是 JSON；`5xx` 回應也不保證是 JSON。解析回應前請先確認 `Content-Type`。

狀態碼

狀態碼	意義	建議處理
<code>200</code> / <code>201</code> / <code>204</code>	成功； <code>204</code> 沒有回應內容，常見於刪除	—
<code>202</code>	已受理但尚未完成，例如同步回覆時附件還在建立索引，回應 <code>status: "pending_attachments"</code>	稍後重試，見 不會進入串流的情況
<code>301</code>	路徑少了結尾的 <code>/</code>	改用帶 <code>/</code> 的網址
<code>400</code>	欄位驗證失敗；資源 ID 不是有效的 UUID 時，多數端點回 <code>{"detail": "Invalid UUID format."}</code>	修正請求，不要重試
<code>401</code>	沒帶憑證或憑證無效，見 認證失敗的回應	檢查金鑰與標頭格式，不要重試
<code>402</code>	方案、Credits 或額度不足，見 下方說明	通知組織管理者
<code>403</code>	角色權限不足、組織未開通該功能、組織已停用，或字數計費方案的用量已達上限	調整角色權限或洽窗口
<code>404</code>	資源不存在、不屬於金鑰所屬的組織，或網址打錯	確認 ID、金鑰所屬組織與路徑
<code>409</code>	與資源目前的狀態衝突，例如同一件工作已在執行中、要刪除的資源仍被引用	讀回最新狀態再決定，不要原樣重送

狀態碼	意義	建議處理
429	超過速率限制	依 <code>Retry-After</code> 退避
5xx	平台或上游服務錯誤	指數退避後重試

409 的回應是 `detail` 物件，部分端點會附上處理所需的欄位，例如知識庫已有 FAQ 匯入在執行時：

```
{"detail": "An FAQ import is already running for this knowledge base.", "jobId": "..."}

```

402：方案、Credits 或額度不足

情況	回應內容	有沒有 <code>code</code>
組織沒有有效的付費方案	<code>{"detail": "Your organization has no active paid plan. ...", "code": "no_active_plan"}</code>	有， <code>no_active_plan</code>
組織 Credits 不足	<code>{"detail": "Insufficient credits ..."} </code> ，文字依端點而異	沒有
成員的個人額度用完或未分配，或所屬角色的額度用完	<code>{"detail": "Personal credit quota exhausted. ..."} </code> ，文字依情況而異	沒有

只有「沒有有效的付費方案」會帶 `code`。Credits 不足與額度用完的回應只有 `detail`，目前無法只從回應內容分辨是組織 Credits 不足，還是個人或角色額度用完，請組織管理者到後台確認。

Credits 與額度在會產生用量的寫入請求檢查，例如送出訊息、知識庫檔案上傳與搜尋、網頁爬取；查詢類的 `GET` 請求不會因此回 402。

AI Gateway 的 OpenAI 相容端點回 OpenAI 格式：`{"error": {"type": "billing_error", "code": "insufficient_credits", ...}}`，沒有有效方案時也是這個 `code`。

平台未啟用 Credits 計費時（部分地端部署），不會出現這類 402。

分頁

多數列表端點用頁碼分頁：

參數	說明
<code>page</code>	頁碼，從 1 開始；超出範圍回 404 <code>{"detail": "Invalid page."}</code>
<code>page_size</code> （也可寫 <code>pageSize</code> ）	每頁筆數，預設 20、上限 100；超過 100 會自動截成 100，不會報錯

回應格式為 `{"count": ..., "next": "...", "previous": "...", "results": [ ... ]}`，取下一頁時直接使用 `next` 的網址，沒有下一頁時為 `null`。

另外兩種情況：

游標分頁：對話列表（`GET /api/v1/conversations/`）與訊息列表（`GET /api/v1/messages/`）用 `cursor` 參數，由新到舊翻頁，以 `next` 取更早的資料。訊息列表的回應帶 `count`，對話列表沒有；訊息列表每一頁內的訊息依時間由舊到新排列。

不分頁：部分清單直接回傳陣列，例如 `GET /api/v1/credit/items/`。

各端點屬於哪一種，以該端點頁的回應範例為準。

欄位格式

命名：回應的 key 一律是 camelCase。請求的 JSON 與查詢參數可以用 camelCase，snake_case 也接受。少數自由格式的欄位內容會原樣保留、不轉換大小寫，例如 JSON Schema、自訂 HTTP 標頭與 metadata。

時間：多數時間欄位輸出為 Unix 時間戳記的毫秒字串，例如 `"1759536000000"`，精度到秒（最後三位固定是 `000`）。少數欄位使用 ISO 8601 字串，以該端點頁為準。

小數：金額、單價等小數欄位以字串輸出，例如 `"316.8000000000"`，請用十進位型別處理，避免浮點誤差。

ID：資源 ID 多為 UUID。

語系：錯誤訊息會依 `Accept-Language` 標頭翻譯（例如 `ja`），沒有翻譯的訊息維持英文，請勿依賴訊息文字。

上傳大小上限

上傳上限由平台設定，可能調整，地端部署也可能不同。請以 `GET /api/v1/allowed-file-specs/` 的回應為準：這支端點不需認證，依用途列出允許的副檔名與 `maxSize`（位元組）。2026 年 10 月正式區的值如下：

用途	上限
知識庫檔案	100 MB
對話附件：文件、試算表、簡報與音檔	10 MB
對話附件：圖片	5 MB
會議記錄音檔	100 MB

以 Presigned 方式上傳時，取得上傳網址的 `POST /api/v1/upload-presigned-url/` 就會依 `fileSize` 與副檔名檢查，超過上限或類型不允許回 `400`。流程見技術人員手冊 [Presigned 檔案上傳模式](#)。

知識庫的中繼資料、標籤與 FAQ 的 Excel 匯入，共用知識庫檔案的上限。

常見問題

- ▶ 金鑰明明是對的，卻一直回 401？
- ▶ 可以限制某把金鑰只能呼叫特定 API 嗎？
- ▶ 為什麼 `/api/` 開頭的範例和 `/api/v1/` 都能用？

狀態碼

在透過 API 送出 HTTP 請求時，可使用以下狀態碼查看 API 請求成功與否：

狀態碼說明

正確代碼

狀態碼	說明
200	請求成功
201	資源成功建立
204	請求成功，沒有回應內容（常見於刪除）

正確回應範例

此處為內容示範，不同端點的回傳格式內容可能有差異，請以該端點的說明文件參考為主

```
{
  "count": "integer",
  "next": "string|null",
  "previous": "string|null",
  "results": [
    // 結果回傳內容
  ]
}
```

錯誤代碼

狀態碼	說明
400	請求格式錯誤
401	認證失敗
402	方案、Credits 或額度不足，見 402 回應
403	權限不足

狀態碼	說明
404	資源不存在
409	與資源目前的狀態衝突，見 409 回應
429	超過速率限制，見 429 回應
500	伺服器錯誤

錯誤回應的內容主要有三種形狀：detail 物件（例如 {"detail": "..."}，少數會多帶 code）、欄位物件（key 是 camelCase 欄位名）、字串陣列；AI Gateway 的 OpenAI 相容端點則使用 OpenAI 格式的 error 物件。處理時請依狀態碼與 code 判斷，不要比對 detail 文字。各狀態碼的完整說明與處理建議見 [API 共通規範：錯誤回應](#)。

錯誤回應範例

400 - 請求錯誤：必填欄位缺失

```
{
  "files": ["This field is required."],
  "filename": ["This field is required."],
  "file": ["This field is required."]
}
```

402 - 方案、Credits 或額度不足

402 有三種情況，只有「沒有有效的付費方案」會帶 `code`：

情況	<code>code</code>
組織沒有有效的付費方案	<code>no_active_plan</code>
組織 Credits 不足	無
成員的個人額度用完或未分配，或所屬角色的額度用完	無

```
{
  "detail": "Your organization has no active paid plan. Please upgrade to continue using the platform.",
  "code": "no_active_plan"
}
```

```
{
  "detail": "Insufficient credits for KB search. Please top up your account."
}
```

Credits 不足與額度用完的回應只有 `detail`，文字依端點與語系而異，無法從回應內容分辨是哪一種，請組織管理者到後台確認。AI Gateway 端點的格式不同，見 [API 共通規範：402](#)。

409 - 與資源目前的狀態衝突

請求與資源目前的狀態衝突，例如同一件工作已在執行中，或要刪除的資源仍被其他資源引用。回應是 `detail` 物件，部分端點會附上處理所需的欄位：

```
{
  "detail": "An FAQ import is already running for this knowledge base.",
  "jobId": "..."
}
```

請先讀回資源的最新狀態再決定下一步，不要原樣重送。

429 - 超過速率限制

超過 API Key 的每分鐘請求數（RPM）或端點本身的固定上限時回 `429`，並帶 `Retry-After` 標頭（秒數）：

```
{
  "detail": "Request was throttled. Expected available in 42 seconds."
}
```

請依 `Retry-After` 的秒數退避後再重試，不要立刻重送。哪些範圍會限流見 [API 共通規範：速率限制](#)。

轉真人 Webhook 事件

對話進入轉真人流程的每個階段——AI 判定需要真人、指派給客服、轉回 AI、排隊逾時——MaiAgent 都會以 HTTPS POST 推送一筆事件到你設定的端點，讓派工、告警、CRM 或你自己的客服系統自動接手，不需輪詢。事件名稱描述轉真人的狀態轉移，是固定的公開契約；每個事件在對話事件紀錄 API（`GET /api/v1/conversations/{id}/events/`）都有對應的 `event_type`（見下表），用於對帳或改以輪詢取得同一份資訊。

啟用方式

由客戶的管理者或串接人員在後台自助完成，毋需工程協助：

後台「客服對話」→「對話平台」→編輯要串接的對話平台→「客服轉真人設定」→「通知」分頁
開啟「Webhook 通知（系統串接）」開關，填入接收端網址（僅接受 `https://` 開頭）
（選填但建議）填入簽章密鑰，啟用來源驗證（見下方「簽章驗證」）

Webhook 與通知中心、Email 是三個獨立開關，互不影響；預設關閉、逐收件匣啟用。此通道與收件匣既有的「訊息 Webhook」（訊息收送）互不相干。

事件種類

event	事件紀錄 API 的 <code>event_type</code>	何時送出	備註
<code>handoff.requested</code>	<code>transfer_triggered</code>	AI 判定需要真人：對話進入排隊、AI 停止自動回覆	每次轉真人流程的第一筆事件； <code>assignee</code> 為 <code>null</code> 。真人客服在你自己的系統時，這就是接手的訊號
<code>handoff.assigned</code>	<code>human_assigned</code>	對話被指派給 MaiAgent 後台的客服成員（含手動指派、自行認領、轉接、自動指派）	<code>assignee</code> 為被指派客服；轉接給另一位客服會再送一筆（事件 <code>id</code> 不同）
<code>handoff.returned_to_ai</code>	<code>returned_to_ai</code>	客服在後台按「轉回 AI Agent」或透過 <code>return-</code>	<code>assignee</code> 為 <code>null</code> （對話已無負責客服）

event	事件紀錄 API 的 event_type	何時送出	備註
		to-ai API 將對話交回 AI	
handoff.queue_timeout	queue_timeout	排隊超過收件匣設定的「等待超時時間」仍無人接手，MaiAgent 自動把對話轉回 AI	assignee 為 null。真人客服在你自己的系統時，收到它代表 MaiAgent 已在你不知情下恢復 AI 回覆——請視為需要告警的異常；或在派工設定選「接手制」並勾選「不逾時」（API 為 dispatchMode: "handover"、assignmentQueueTimeout: 0；自動派工模式下送 0 會被改成 300 秒）

四種事件的 payload 鍵集、簽章與重試行為完全相同，接收端依 event 欄位分流；未來可能新增事件種類，遇到不認識的 event 請回 2xx 並忽略，不要當成錯誤。

handoff.returned_to_ai 與 handoff.queue_timeout 都代表「AI 恢復自動回覆」，差別在前者是人（專員或你的系統）主動、後者是逾時自動。兩者都不等於「對話已結案」：客服直接把對話標為「已解決」、或客服因失去收件匣權限而使對話重新排隊時，不會送出事件。需要完整的結案訊號時，請以對話 API 查詢 status，或在訊息 Webhook 的 conversation.progress_status 觀察狀態變化。

事件格式

```
POST <你設定的網址>
Content-Type: application/json
```

```
{
  "event": "handoff.assigned",
  "id": "9f0e1d2c- ... ",
  "occurred_at": "2026-09-09T02:30:00.123456+00:00",
  "conversation": {
    "id": "6c2b7c1e- ... ",
    "url": "https://admin.maiagent.ai/ ... "
  },
  "inbox": { "id": "0f9a3d42- ... " },
  "organization": { "id": "b1d20c77- ... " },
  "contact": { "id": "a4c1f6b3- ... " },
  "assignee": { "id": "3e8f5a90- ... " }
}
```

欄位	類型	說明
event	string	事件名稱（見上方「事件種類」）
id	string (uuid)	事件識別碼，與事件紀錄 API 回傳的 <code>id</code> 相同；重試送出的內容完全一樣， 以它去重
occurred_at	string (date-time)	事件發生時間（ISO 8601）；推送不保證依序到達， 以它排序
conversation.id	string (uuid)	對話識別碼
conversation.url	string (uri)	後台對話頁連結
inbox.id	string (uuid)	收件匣識別碼
organization.id	string (uuid)	組織識別碼
contact	object	null
assignee	object	null

欄位命名與巢狀結構和收件匣「訊息 Webhook」的 MaiAgent 標準格式一致（snake_case、關聯物件以 `{ "id" }` 巢狀），同一支接收端可以同時處理兩組 Webhook。Payload 只含識別資訊、事件時間與後台連結，**不含對話文字內容與轉真人理由**；需要對話內容時，請以識別碼呼叫對話與訊息 API 查詢。

簽章驗證（建議啟用）

在後台設定簽章密鑰後，每次推送會帶下列標頭：

```
X-MaiAgent-Signature-256: sha256=
```

值為以密鑰對 **原始 request body bytes** 計算的 HMAC-SHA256 十六進位摘要。驗證時務必使用收到的原始 bytes，不要先解析再重新序列化（欄位順序與空白差異會使摘要不一致）。

```
import hashlib
import hmac

def verify_signature(raw_body: bytes, header_value: str, secret: str) → bool:
    expected = 'sha256=' + hmac.new(secret.encode(), raw_body,
    hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)
```

密鑰為 write-only：儲存後不回顯，後台僅顯示「已設定」；欄位留空儲存代表維持既有密鑰，更換直接輸入新值，移除請使用「清除密鑰」操作。未設定密鑰時，推送不帶簽章標頭。

接收端最小範例

以下是一支可直接啟動的 Python（Flask）接收端：讀取原始 bytes 驗章、依 `event` 分流、先回 2xx 再做耗時處理。把 `HANDOFF_WEBHOOK_SECRET` 換成後台設定的簽章密鑰即可。

```

import hashlib
import hmac
import os

from flask import Flask, abort, request

app = Flask(__name__)
SECRET = os.environ["HANDOFF_WEBHOOK_SECRET"]

def verify_signature(raw_body: bytes, header_value: str | None) → bool:
    if not header_value:
        return False
    expected = "sha256=" + hmac.new(SECRET.encode(), raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)

@app.post("/webhooks/handoff")
def handoff_webhook():
    raw = request.get_data() # 原始 bytes，不要用 request.json 再序列化
    if not verify_signature(raw, request.headers.get("X-MaiAgent-Signature-
256")):
        abort(401)

    event = request.get_json(force=True)
    if already_processed(event["id"]): # 重試會送完全相同的內容，以事件 id 去重
        return "", 204

    conversation_id = event["conversation"]["id"]
    if event["event"] == "handoff.requested":
        # AI 已停止回覆；真人客服在你的系統時，在這裡開單、派客服
        open_ticket(conversation_id, event["conversation"]["url"])
    elif event["event"] == "handoff.assigned":
        # 指派給 MaiAgent 後台的客服；轉接給另一位客服時會再收到一筆，assignee 會變
        assign_ticket(conversation_id, event["assignee"]["id"])
    elif event["event"] == "handoff.returned_to_ai":
        close_ticket(conversation_id)
    elif event["event"] == "handoff.queue_timeout":
        # MaiAgent 已自動恢復 AI 回覆；若你的客服仍在處理，這是需要告警的異常
        close_ticket(conversation_id, reason="queue_timeout")
    # 未知事件：回 2xx 忽略，不要當錯誤

    return "", 204 # 10 秒內回 2xx；耗時工作丟到背景

def already_processed(event_id):

```

```

...

def open_ticket(conversation_id, url):
    ...

def assign_ticket(conversation_id, assignee_id):
    ...

def close_ticket(conversation_id, reason=None):
    ...

```

```
HANDOFF_WEBHOOK_SECRET=你的密鑰 flask --app receiver run --port 8080
```

本機測試可用下列指令模擬一筆推送（簽章用同一把密鑰算）：

```

BODY='{ "assignee":{ "id": "3e8f5a90-0000-0000-0000-000000000004"}, "contact":
{ "id": "a4c1f6b3-0000-0000-0000-000000000005"}, "conversation":{ "id": "6c2b7c1e-
0000-0000-0000-
000000000001", "url": "https://admin.maiagent.ai/conversations/index?
conversationId=6c2b7c1e-0000-0000-0000-
000000000001"}, "event": "handoff.assigned", "id": "9f0e1d2c-0000-0000-0000-
000000000006", "inbox":{ "id": "0f9a3d42-0000-0000-0000-
000000000002"}, "occurred_at": "2026-09-09T02:30:00.123456+00:00", "organization":
{ "id": "b1d20c77-0000-0000-0000-000000000003"} }'
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "你的密鑰" | awk '{print
$NF}')
curl -X POST http://localhost:8080/webhooks/handoff
-H "Content-Type: application/json"
-H "X-MaiAgent-Signature-256: sha256=$SIG"
-d "$BODY"

```

回應與重試

接收端應在 10 秒內以 2xx 狀態碼回應，回應內容不拘。

逾時、連線失敗或非 2xx 回應時，系統會自動重試，最多 3 次、間隔 60 秒；重試送出的內容與原始推送完全相同（同一個 `id`），接收端請以事件 `id` 去重。事件各自以背景任務送出，`handoff.requested` 與緊接著的 `handoff.assigned` 可能不依序到達，請以 `occurred_at` 排序。

Webhook 推送失敗不影響轉真人主流程，也不影響通知中心與 Email 通知。

轉真人後由外部系統接手對話

若你的真人客服不在 MaiAgent 後台操作，而是在自己的客服系統回覆，可以把本頁的事件與收件匣既有的「訊息 Webhook」、訊息 API 組合起來，讓對話在轉真人期間雙向同步。所有步驟都用既有能力，不需要額外開通。

若訪客的通道（網頁、LINE 等）本來就掛在你的系統、只把 MaiAgent 當 AI 引擎，接入方式請改看技術文件的第三方客服系統串接（含轉真人）；本節針對的是訪客在 MaiAgent 的通道上、只有真人客服在你系統的情境。

串接流程

得知轉真人開始：收到 `handoff.requested` 後，在你的系統以 `conversation.id` 開單或建立會話，並記下 `contact.id`；之後的 `handoff.assigned` 帶 `assignee.id`，可用來更新負責人。派工設定請選「接手制」並勾選「不逾時」（自動派工模式不能不逾時，送 0 會被改成 300 秒），否則逾時會收到 `handoff.queue_timeout`、AI 自動恢復回覆。

接收客戶訊息與客服回覆：在後台該收件匣的「訊息 Webhook」新增 `incoming`（客戶訊息）與 `outgoing`（AI 或客服回覆）方向的 Webhook。每筆訊息的 `conversation` 物件帶有目前處理狀態，只處理 `progress_status` 為 `human_serving` 的訊息即可略過 AI 模式的流量：

欄位	說明
<code>conversation.status</code>	<code>open</code> <code>queued</code> <code>resolved</code>
<code>conversation.progress_status</code>	<code>ai_processing</code> <code>waiting_for_human</code> <code>human_serving</code> <code>resolved</code> <code>open</code>
<code>conversation.auto_reply_enabled</code>	AI 是否會自動回覆；轉真人後為 <code>false</code>
<code>conversation.assignee</code>	負責客服 <code>{ "id", "name" }</code> ，無人負責時為 <code>null</code>
<code>sender.type</code>	<code>contact</code> （客戶） <code>user</code> （真人客服） <code>chatbot</code> （AI）

把客服回覆寫回對話：以 API 金鑰呼叫 `POST /api/v1/messages/outgoing/`，帶 `conversation` 與 `content`，MaiAgent 會把訊息轉發到客戶所在的渠道。訊息的發送者為該 API 金鑰所屬的成員，該成員需具備此收件匣的存取權限（詳見「對話與訊息」端點說明）。

結束轉真人：客服處理完畢後呼叫 `PATCH /api/v1/conversations/{id}/return-to-ai/`，AI 重新接手；無論是你呼叫 API 還是客服在 MaiAgent 後台按「轉回 AI Agent」，你都會收到 `handoff.returned_to_ai`，請據以關閉你系統中的會話。

呼叫範例

(1) 轉真人期間收到的訊息 Webhook (`maiaagent_standard` 格式，節錄) —— 用 `conversation.progress_status` 判斷是否已在真人模式、`sender.type` 判斷發話者：

```
{
  "id": "9d2f7a10- ... ",
  "type": "incoming",
  "content": "我要改訂單地址",
  "created_at": "2026-09-09T02:30:00+00:00",
  "conversation": {
    "id": "6c2b7c1e- ... ",
    "title": " ... ",
    "type": "individual",
    "status": "open",
    "progress_status": "human_serving",
    "auto_reply_enabled": false,
    "assignee": { "id": "3e8f5a90- ... ", "name": "客服小美" }
  },
  "inbox": { "id": "0f9a3d42- ... ", "name": "官網客服", "channel_type": "web" },
  "contact": { "id": "a4c1f6b3- ... ", "name": "王小明", "email": null,
"phone_number": null },
  "sender": { "id": "a4c1f6b3- ... ", "name": "王小明", "type": "contact" },
  "attachments": [],
  "metadata": {}
}
```

(2) 把客服回覆寫回對話 —— `conversation` 與 `content` 即可，其餘欄位選填；訊息會以 API 金鑰所屬成員的身分送到客戶所在渠道：

```
curl -X POST "https://api.maiagent.ai/api/v1/messages/outgoing/"
-H "Authorization: Api-Key YOUR_API_KEY"
-H "Content-Type: application/json"
-d '{
  "conversation": "6c2b7c1e- ... ",
  "content": "您好，地址已為您更新為台北市信義區..."
}'
```

(3) 處理完畢，把對話交回 AI——不需要 request body，成功回 200 與對話物件；之後 AI 恢復自動回覆、`assignee` 清空：

```
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/6c2b7c1e- ... /return-to-ai/"
-H "Authorization: Api-Key YOUR_API_KEY"
```

三步合在一起 (Python, `requests`)：

```
import requests

API = "https://api.maiagent.ai/api/v1"
HEADERS = {"Authorization": "Api-Key YOUR_API_KEY"}

def on_message_webhook(payload: dict) → None:
    conv = payload["conversation"]
    if conv["progress_status"] ≠ "human_serving":
        return # AI 模式的流量不進客服系統
    if payload["sender"]["type"] = "contact":
        push_to_agent_console(conv["id"], payload["content"]) # 客戶訊息→你的客服介面

def reply(conversation_id: str, text: str) → None:
    requests.post(f"{API}/messages/outgoing/", headers=HEADERS,
                  json={"conversation": conversation_id, "content": text},
                  timeout=10).raise_for_status()

def finish(conversation_id: str) → None:
    requests.patch(f"{API}/conversations/{conversation_id}/return-to-ai/",
                  headers=HEADERS, timeout=10).raise_for_status()
```

注意事項

透過 API 寫回的回覆會再經由 `outgoing` 訊息 Webhook 外送一次 (`sender.type` 為 `user`)；請以訊息 `id` 去重，避免在你的系統重複顯示。

`handoff.assigned` 在轉接給另一位客服時會再送一筆，`assignee.id` 為新的負責人；請以 `conversation.id` 更新既有會話而非重複開單。

訊息 Webhook 與轉真人 Webhook 是兩組獨立設定與簽章方式：前者以 `X-Webhook-Secret` 標頭原樣帶出設定的密鑰，後者為本頁的 HMAC-SHA256 簽章。

客戶端工具 (Client Tools)

客戶端工具讓 AI 助理可以呼叫「在你的應用程式裡執行」的工具。你在每次呼叫對話 API 時宣告工具清單；模型判斷需要用到某個工具時，MaiAgent 不會自己執行，而是把「要呼叫哪個工具、帶什麼參數」回傳給你，由你的應用程式在本機執行後，再把結果送回對話，模型會依結果繼續回答或呼叫下一個工具。

適合只有你的應用程式才能執行的動作，例如查詢只在公司內網可達的系統、瀏覽器擴充功能操作目前開啟的網頁、App 存取使用者裝置上的資源。

使用情境

一家電商的工程師把 AI 助理接進公司內部的客服工作台，訂單系統只開放在內網。客服人員在工作台輸入「客人問訂單 A1024 什麼時候到貨」，工作台呼叫對話 API 時一併宣告了「查詢訂單」工具。模型回覆要查哪一筆訂單，工作台在內網查到出貨狀態後送回同一段對話，模型據此寫出回覆客人的說明。訂單系統不必對外開放，工作台只送出模型需要的資料。

使用條件

客戶端工具已對所有組織開放，不需要另外申請。要讓模型能呼叫，需同時符合：

AI 助理的 **回答模式** 為 **Agent 模式**

AI 助理使用的模型支援工具呼叫

不符合上述條件時，API 不會回傳錯誤，而是直接忽略你宣告的客戶端工具，模型會在沒有這些工具的情況下回答。若模型始終不呼叫你的工具，請先檢查助理的回答模式與模型。

運作流程

宣告工具並送出訊息

呼叫 `POST /api/v1/chatbots/{chatbotId}/completions/`，在 `message.clientTools` 帶入工具定義。

收到工具呼叫要求

模型決定呼叫你的工具時，回應的 `contentPayload.items` 會出現 `type` 為 `client_tool_call` 的項目，**本回合隨即結束**。

在你的應用程式執行工具

依 `name` 與 `input` 執行對應的工具。

把結果送回同一段對話

以同一個 `conversation` 再呼叫一次對話 API，把執行結果寫在 `message.content`，並再次帶上 `clientTools`。模型會接續回答；若還需要其他工具，會再回到步驟 2，直到回傳最終文字為止。

宣告工具

```
{
  "conversation": "<對話 ID，第一則訊息可省略>",
  "isStreaming": false,
  "message": {
    "content": "客人問訂單 A1024 什麼時候到貨？",
    "clientTools": [
      {
        "name": "get_order",
        "description": "依訂單編號查詢訂單的出貨狀態與預計到貨日",
        "inputSchema": {
          "type": "object",
          "properties": {
            "order": { "type": "string", "description": "訂單編號" }
          },
          "required": ["order"]
        }
      }
    ]
  }
}
```

欄位	類型	說明
name	string	工具名稱，僅限英數字、底線 <code>_</code> 、連字號 <code>-</code> ，長度 1-64，同一次請求內不可重複
description	string	工具用途說明，模型依此判斷何時使用，請寫清楚
inputSchema	object	工具參數的 JSON Schema，序列化後不可超過 4,000 字元

每次請求最多宣告 20 個客戶端工具。工具清單只在該次請求有效，不會記在對話上。後續每一則訊息（包含送回工具結果的訊息）都要重新帶上 `clientTools`，否則模型在那一回合就無法再呼叫。

工具呼叫要求的格式

模型要求呼叫工具時，`contentPayload.items` 會包含如下項目，一次可能有多個：

```
{
  "type": "client_tool_call",
  "toolCallId": "call_abc123",
  "name": "get_order",
  "input": { "order": "A1024" },
  "startTimestamp": "2026-10-01T08:30:00.000000+00:00"
}
```

欄位	類型	說明
type	string	固定為 <code>client_tool_call</code>
toolCallId	string	null
name	string	要執行的工具名稱，對應你宣告的 <code>name</code>
input	object	模型帶入的參數，對應你宣告的 <code>inputSchema</code>
startTimestamp	string	null

非串流與串流的差異

非串流（`isStreaming: false`）：工具呼叫項目在回應本體（`done: true`）的 `contentPayload.items` 中。

串流（`isStreaming: true`）：工具呼叫項目會以獨立事件送出，該事件 `done` 為 `false`、`content` 為空字串、`contentPayload.items` 內含 `client_tool_call`。最後的 `done: true` 事件不會再重複這些項目，請在串流過程中收集。

回應的欄位名稱一律轉為 camelCase，input 內的參數名稱也會被轉換。例如你在 inputSchema 定義的 order_id，在 input 中會變成 orderId。建議參數名稱使用單一小寫英文單字（如 order、keyword），或在解析時同時接受兩種寫法。

送回執行結果

目前沒有專用的「提交工具結果」端點。請在 **同一段對話** 送出下一則一般訊息，把結果寫在 `message.content`：

```
{
  "conversation": "<同一個對話 ID>",
  "isStreaming": false,
  "message": {
    "content": "工具 get_order (toolCallId: call_abc123) 執行結果：\n<查詢結果>",
    "clientTools": [ " ... 與前一次相同的工具定義 ... " ]
  }
}
```

一次多個工具呼叫：全部執行完後，把各工具的結果合併在同一則訊息送回。

寫上工具名稱與 `toolCallId`：後端不會用 `toolCallId` 比對，但寫在結果文字中能幫助模型對應是哪一次呼叫。

執行失敗或使用者拒絕：同樣用文字說明，例如「get_order 執行失敗：查無此訂單」或「使用者拒絕執行此操作」，模型會據此調整回答或改用其他方式。

結果內容長度：送回的內容會計入模型輸入，資料量大時請先截取需要的部分。

連續呼叫多個工具

每一次「模型要求呼叫 → 你送回結果」就是一個回合。模型可以在多個回合中連續呼叫工具，直到回應中不再出現 `client_tool_call`、改為回傳最終文字。建議在你的應用程式設定回合上限，避免異常情況下無限循環。

常見問題

- ▶ 模型一直沒有呼叫我的工具
- ▶ 送回結果後模型說無法使用工具
- ▶ 收到 400 錯誤